

---

---

A PREPRINT

**Artificial Intelligence**

April 28, 2026

# [Gemini API Error: 429 You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google.dev/gemini-api/docs/rate-limits>. To monitor your current usage, head to: <https://ai.dev/rate-limit>. \* Quota exceeded for metric: generativelanguage.googleapis.com/generateContent:tier\_equest\_limit: 20, model : gemini - 2.5 - flashPlease retry in 20.777327173s. [linksdescription : "The field of neural network AI has undergone significant evolution, with Convolutional Neural Networks (CNNs) emerging as a dominant paradigm for visual data analysis. More recently, alternatives to the ubiquitous backpropagation algorithm, such as the Forward-Forward (FF) algorithm, have garnered attention for their potential biological plausibility and computational efficiencies. This section reviews the foundational principles of CNNs, their architectural advancements and applications, the development of alternative training mechanisms, and the recent exploration of the Forward-Forward algorithm, ultimately identifying the gaps addressed by the current work." ]

## AI-Generated Survey Paper • Gemini 1.5 Flash + LlamaParse

## Abstract [Gemini API Error: 429 You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google.dev/gemini-api/docs/rate-limits>. To monitor your current usage, head to: <https://ai.dev/rate-limit>. \* Quota exceeded for metric: generativelanguage.googleapis.com/generateContent:tier\_equest\_limit: 5, model : gemini - 2.5 - flashPlease retry in 3.84441269s. [linksdescription : "The field of neural network AI has undergone significant evolution, with Convolutional Neural Networks (CNNs) emerging as a dominant paradigm for visual data analysis. More recently, alternatives to the ubiquitous backpropagation algorithm, such as the Forward-Forward (FF) algorithm, have garnered attention for their potential biological plausibility and computational efficiencies. This section reviews the foundational principles of CNNs, their architectural advancements and applications, the development of alternative training mechanisms, and the recent exploration of the Forward-Forward algorithm, ultimately identifying the gaps addressed by the current work." ]

## Introduction Convolutional Neural Networks (CNNs) have emerged as a foundational paradigm in modern machine learning, delivering remarkable performance across a myriad of complex tasks, from image classification [1, 11] and object detection [?] to medical image analysis [?] and document recognition [?]. Inspired by the biological visual cortex, these networks excel at automatically learning hierarchical, spatially invariant features directly from raw data, thereby revolutionizing the field by minimizing the need for handcrafted feature engineering. The impressive capabilities of CNNs are largely attributed to their training methodology, which predominantly relies on the backpropagation algorithm [Rumelhart et al., 1986] for efficient weight optimization through gradient descent. However, despite its widespread success, backpropagation inherently presents several computational and conceptual challenges that warrant exploration into alternative training strategies.

The standard backpropagation algorithm imposes significant demands on computational resources and memory, particularly as CNN architectures grow deeper and input resolutions increase. Storing intermediate activations for the backward pass can lead to substantial memory allocation problems, making the training of high-capacity models or processing large images (e.g., ImageNet at  $227 \times 227$ ) exceedingly resource-heavy [?]. Beyond these practical limitations, the biological plausibility of backpropagation, which necessitates precise symmetric forward and backward passes, has been a subject of ongoing debate within the neuroscience and AI communities [Lillicrap et al., 2020]. These challenges have motivated a quest for novel, more efficient, and potentially biologically inspired learning rules that can circumvent backpropagation’s drawbacks. In this context, Geoffrey Hinton’s recently introduced Forward-Forward (FF) algorithm [Hinton, 2022] stands out as a promising, backpropagation-free approach to neural network training.

This survey paper provides a comprehensive review of the Forward-Forward algorithm and its evolving applications to Convolutional Neural Networks. We begin by delving into the theoretical foundations of the FF algorithm, elucidating its core principles of layer-wise, local learning through “goodness” functions, and contrasting it with the global error propagation of backpropagation. Subsequently, we meticulously examine various architectural adaptations and training strategies that utilize FF to effectively train CNNs across diverse tasks. Furthermore, this paper analyzes the empirical performance, computational efficiency, and robustness of FF-trained CNNs compared to their backpropagation-trained counterparts, drawing insights from the burgeoning body of research. We conclude by discussing the current limitations, identifying key open research questions, and outlining promising future directions for backpropagation-free training of convolutional neural networks.

## ## Related Work ## Related Work

The field of neural network AI has undergone significant evolution, with Convolutional Neural Networks (CNNs) emerging as a dominant paradigm for visual data analysis. More recently, alternatives to the ubiquitous backpropagation algorithm, such as the Forward-Forward (FF) algorithm, have garnered attention for their potential biological plausibility and computational efficiencies. This section reviews the foundational principles of CNNs, their architectural advancements and applications, the development of alternative training mechanisms, and the recent exploration of the Forward-Forward algorithm, ultimately identifying the gaps addressed by the current work.

## ### Foundational Concepts and Early Development of Convolutional Neural Networks

The concept of neural networks for pattern recognition dates back decades, with early reviews highlighting their potential in image processing [?]. However, modern CNNs trace their lineage primarily to the seminal work of LeCun and colleagues. LeCun et al. [?] demonstrated the effectiveness of backpropagation applied to a multi-layer neural network for handwritten zip code recognition in 1989, laying the groundwork for spatial invariance through shared weights. This was further elaborated in 1998 with their comprehensive work on gradient-based learning applied to document recognition, showcasing CNNs’ ability to learn hierarchical features with local receptive fields and weight sharing [?]. These early architectures, often referred to as LeNet-5, proved highly effective for tasks like handwritten digit classification (e.g., MNIST dataset [LeCun and Cortes, 2010]) and established the core components of modern CNNs: convolutional layers, pooling layers, and fully connected layers. Further evaluations confirmed the efficacy of CNNs for visual recognition tasks [?], and “best practices” for their application to visual document analysis began to emerge [?]. These early contributions underscored the power of exploiting knowledge about the specific type of input, such as images, to simplify network architecture and en-

hance performance compared to traditional Artificial Neural Networks (ANNs).

### ### Advancements in CNN Architectures and Training Techniques

While the fundamental CNN architecture was established early on, significant advancements in network depth, training strategies, and regularization techniques propelled CNNs to state-of-the-art performance across diverse computer vision tasks. Early work by Ciresan et al. pioneered flexible, high-performance convolutional networks for image classification, often employing multi-column deep neural networks and committees of CNNs to achieve superior results in challenging benchmarks [1, 3, 4]. A watershed moment arrived with Krizhevsky et al.’s AlexNet [? ], which dramatically improved ImageNet classification performance in 2012 by employing a deeper architecture, ReLU activations, and crucial regularization techniques like dropout.

The development of robust training techniques was critical for scaling CNNs. Geoffrey Hinton’s work on Restricted Boltzmann Machines (RBMs) provided insights into unsupervised pre-training for deep networks [? ]. More directly, dropout, introduced by Hinton et al. [? ] and further elaborated in Srivastava’s thesis [? ], became a standard regularization technique to prevent co-adaptation of feature detectors, significantly improving generalization. Architectural principles also evolved; for instance, the practice of stacking multiple convolutional layers before pooling layers became common, allowing the network to extract more complex features while maintaining spatial resolution [e.g., as illustrated in the provided architectural description]. The efficient management of computational resources became paramount, leading to suggestions for splitting large convolutional layers into smaller ones to reduce complexity and employing specific input dimensions and zero-padding strategies to control feature map sizes. Visualizing and understanding the features learned by these deep networks, as demonstrated by Zeiler and Fergus [? ], further aided in architectural design and debugging. Similarly, stochastic pooling was proposed as another regularization method for deep CNNs [? ]. Despite these advances, the inherent resource intensiveness of CNNs, particularly for large input images, remained a significant challenge, often necessitating compromises between computational load and model capacity [as discussed in the provided text].

### ### Applications of CNNs in Computer Vision and Medical Imaging

The versatility and effectiveness of CNNs led to their widespread adoption across various computer vision domains. Beyond general image classification [1, 3, 11], CNNs demonstrated success in specific object recognition tasks, such as pedestrian detection [? ] and face detection [? ]. Their ability to process spatial hierarchies of features made them ideal for tasks involving complex visual patterns. For instance, Cireřan et al. applied deep neural

networks to medical image analysis, achieving remarkable results in mitosis detection in breast cancer histology images, a crucial step for cancer diagnosis and grading [? ]. This highlighted the potential of CNNs in specialized domains where intricate visual patterns hold diagnostic significance, a field further reviewed by Gurcan et al. [? ] and Shmatko et al. [? ]. The architecture was also extended to handle temporal data, with 3D Convolutional Neural Networks being developed for human action recognition in videos [? ] and large-scale video classification [? ]. The ability of CNNs to learn invariance properties to transformations like translations and small diffeomorphisms, crucial for natural images [as detailed in the provided definitions of symmetries], underpins their robust performance in these diverse applications. Rawat and Wang [? ] provide a comprehensive review of deep CNNs for image classification, summarizing the advancements and applications in the field.

### ### Alternatives to Backpropagation and Biologically Plausible Learning

Despite the empirical success of backpropagation (BP) in training deep neural networks [Rumelhart et al., 1986], its biological plausibility has been questioned [Lillicrap et al., 2020]. The need for symmetric forward and backward weights, and the instantaneous global availability of error signals, are often cited as challenges in aligning BP with neural mechanisms in the brain. This has spurred research into alternative training algorithms that operate without an explicit backward pass or that are more biologically inspired. Approaches like Direct Feedback Alignment (DFA) [Nøkland, 2016] demonstrated that learning could occur even with fixed random feedback weights, challenging the strict requirement for symmetric weights. Equilibrium Propagation [Scellier and Bengio, 2017] offered another biologically plausible framework by framing learning as an energy minimization problem without explicit gradient computation. More recent efforts have explored various forms of “gradients without backpropagation” [Baydin et al., 2022], often driven by considerations for on-device learning, hardware implementation, or closer alignment with neurobiological principles [Christensen et al., 2022]. Techniques like Multiplexed Gradient Descent [McCaughan et al., 2023] and tensor-compressed back-propagation-free training [Zhao et al., 2023b] further exemplify this trend towards more efficient or biologically inspired learning. Concepts like activation learning by local competitions [Zhou, 2022] also represent a departure from global error signals.

### ### The Rise of the Forward-Forward Algorithm (FF) and its Applications

A significant recent development in this area is the Forward-Forward (FF) algorithm, introduced by Hinton [? ]. FF replaces the single forward and single backward pass of backpropagation with two forward passes per layer: one for a “positive” (real data) input and one for a “negative” (generated or corrupted data) input. Each layer then independently trains its weights to increase the

”goodness” (e.g., sum of squared activations) for positive data and decrease it for negative data. This local, layer-wise learning rule, which eliminates the need for a backward pass, offers potential advantages in terms of computational efficiency, memory usage (as it avoids storing activations for the backward pass), and biological plausibility.

Since its introduction, the FF algorithm has sparked considerable research. Initial implementations and studies focused on fully connected networks (e.g., [Loewe, 2023], which serves as a starting point for this work). Researchers have explored various extensions and applications, including its use in self-supervised learning [Brenig and Timofte, 2023], fine-tuning large language models with only forward passes [Malladi et al., 2023], and its deployment on resource-constrained devices like microcontrollers (e.g.,  $\mu$ -FF [De Vita et al., 2023]). Investigations have also delved into the theoretical aspects of FF, such as the emergence of sparsity [Yang, 2023] and the role of layer collaboration [Lorberbom et al., 2023]. Other related efforts include ”error-driven input modulation” [Dellafrera and Kreiman, 2022], ”Hebbian deep learning without feedback” [Journé et al., 2023], and ”symmetric backpropagation-free contrastive learning” [Lee and Song, 2023], all exploring learning without explicit backpropagation. The algorithm has also been adapted for different network types, such as Graph Neural Networks [Paliotta et al., 2023], and even explored in optical neural networks [Oguz et al., 2023]. Preliminary studies have begun to explore FF for specific image analysis tasks, such as hyperspectral image classification [Paheding and Reyes-Angulo, 2023] and skin lesion classification [Reyes-Angulo and Paheding, 2023], often using it as a feature extractor.

#### ### Gaps and Contribution of the Current Work

While the Forward-Forward algorithm has shown promising results in various contexts, particularly for fully connected networks and as a feature extractor, its comprehensive application and optimization for **Convolutional Neural Networks (CNNs)** for image classification tasks remain an active and crucial area of research. Much of the early FF work, including the foundational implementation by Loewe [? ], primarily focused on fully-connected Deep Neural Networks (DNNs). The distinct architectural characteristics of CNNs—local receptive fields, weight sharing, and pooling layers—introduce unique considerations for how the FF algorithm’s local learning rules interact with these spatial operations. The provided context highlights that CNNs are ”extremely powerful machine learning algorithms” but can be ”horrendously resource-heavy,” especially for large images, and intricate architectural choices regarding filter sizes, strides, and padding significantly impact performance and memory.

This work addresses the gap by investigating the effective implementation and training of CNNs using the Forward-Forward algorithm, moving beyond simple fully-connected architectures or FF as merely a feature ex-

tractor. Specifically, by adapting and extending Loewe’s [? ] FF-trained fully connected DNN code, this research aims to provide a concrete framework and evaluation of FF-trained CNNs for image analysis. The exploration of different inference methods for FF, such as the linear classifier and goodness evaluation, also contributes to understanding how to best leverage FF-trained networks. The challenges related to memory allocation and computational complexity, which are inherent to deep CNNs, are re-evaluated within the context of the FF algorithm, potentially revealing new trade-offs or efficiencies. By systematically applying and evaluating FF within the CNN paradigm, this work contributes to understanding its potential as a viable alternative to backpropagation for complex visual recognition tasks, bridging the gap between the robust performance of CNNs and the appealing properties of backpropagation-free learning.

#### ## Methodology Overview ## Methodology Overview

Our methodology centers on the application and training of Convolutional Neural Networks (CNNs), a class of biologically inspired neural networks particularly effective for image analysis and other machine learning problems. CNNs leverage a hierarchical architecture to extract increasingly complex features from input data, making them highly suitable for tasks requiring robust feature representations.

##### ### Convolutional Neural Network Architecture

A typical CNN architecture processes an input signal,  $x$ , through a series of layers. Each subsequent layer,  $x_j$ , is computed by applying a linear operator  $W_j$  (often a convolution) followed by a non-linear activation function  $\rho$ . Common non-linearities include the Rectified Linear Unit (ReLU), defined as  $\max(x, 0)$ , or a sigmoid function. This hierarchical structure allows the network to learn progressively abstract representations.

The core operations include:

- Convolutional Layers:** *These layers apply a stack of convolutional filters (kernels) to the input, producing feature maps. The discrete convolution operator is defined as  $(fg)(x) = \sum_{u=-\infty}^{\infty} f(u)g(x-u)$ .*
- Non-linear Activation Functions:** *Applied after each convolutional operation, these functions introduce non-linearity, enabling the network to model complex relationships.*
- Pooling Layers:** *Often included after convolutional layers, pooling operations (e.g., max pooling) reduce the spatial dimensions of the feature maps, which helps in achieving translational invariance and reducing computational load. However, in some architectures, pooling layers may be omitted if found to decrease accuracy.*
- Fully-Connected Layers:** *After several convolutional and pooling stages, the processed feature maps are typically flattened and fed into one or more fully-connected layers, which perform high-level reasoning and classification.*

A key advantage of CNNs is their ability to learn invariant feature transformations, especially with respect to local symmetries such as translations and diffeomorphisms

present in natural images (e.g., variations in pose, lighting, or scale). The goal is to construct features that suppress intra-class variations while preserving inter-class distinctions.

### ### Training and Optimization

The optimization problem associated with training CNNs is inherently non-convex. Typically, the weights of the network are learned using **stochastic gradient descent (SGD)**, with gradients computed efficiently via the **back-propagation algorithm**.

Our study also explores specialized training paradigms, such as **Forward-Forward (FF) training**. In this approach, individual layers of the CNN are trained sequentially. The loss for each layer is evaluated by computing the sigmoidal function of a "goodness" parameter, which is defined as the sum of squared layer activations, modified by subtracting a layer-specific threshold  $\theta$ . This loss calculation is differentiated for positive and negative data samples to promote discriminative feature learning within each layer. Cumulative network loss, summing individual layer losses, is employed to foster collaboration between layers.

To enhance training stability and performance, several techniques are applied: \* **Layer Normalization**: Applied between individual layers, it normalizes activations to ensure subsequent layers primarily learn patterns rather than scale. \* **Learning Rate Schedule**: The learning rate is dynamically adjusted during training, for instance, by employing a linear cooldown schedule halfway through the training epochs.

### ### Inference

For classification tasks, two primary inference methods are commonly used: \* **Linear Classifier**: The default method involves connecting the neurons of every layer (except the first) to an output layer with a number of nodes equal to the number of classes. The connecting weights are trained using a cross-entropy loss, and the network directly outputs class probabilities or scores. \* **Goodness Evaluation**: This method, particularly relevant for FF-trained networks, involves exposing the image multiple times, each time superimposed with one of the possible labels. A "goodness parameter" (e.g., sum of squared activations) is computed for each label, and the image is assigned the label characterized by the highest goodness value.

### ### Architectural Design Principles

While there is no rigid blueprint for CNN architecture, common practices and "rules-of-thumb" have emerged: \* **Layer Stacking**: Stacking multiple convolutional layers (e.g., two) before a pooling layer is often recommended to allow for the extraction of more complex features. \* **Filter Size and Stride**: Using small filters (e.g.,  $3 \times 3$ ) with a stride of one and appropriate zero-padding is often advised to maintain spatial dimensionality and capture fine-grained features. For larger images or resource-

constrained scenarios, larger filter sizes with increased stride (e.g., 2) may be considered. \* **Input Dimensions**: Input images are often scaled to dimensions recursively divisible by two (e.g.,  $32 \times 32$ ,  $224 \times 224$ ). \* **Computational Efficiency**: To manage the substantial computational and memory demands of CNNs, especially with large images, techniques include splitting large convolutional layers into multiple smaller ones (e.g., three  $3 \times 3$  layers instead of one  $7 \times 7$ ) to reduce parameters and increase non-linearity, or resizing raw input images.

Our implementation leverages Python with the PyTorch library, building upon existing frameworks for FF-trained networks and running on high-performance computing resources. All results are typically validated using standard datasets such as MNIST, with training and validation splits optimized for hyperparameter search.

## ## Artificial intelligence ## Artificial intelligence

### ### Convolutional Neural Networks

Convolutional Neural Networks (CNNs) constitute a distinct class of deep learning models specifically engineered to process data characterized by a grid-like topology, most notably images [? ]. Their architectural design is fundamentally optimized to exploit the spatial relationships and local patterns inherent in visual information.

#### #### Core Architectural Principles

CNNs diverge from standard Artificial Neural Networks (ANNs) through several key architectural distinctions. A central feature is the three-dimensional organization of neurons within their layers. Neurons are arranged across two spatial dimensions, corresponding to the height and width of the input, and a third dimension referred to as 'depth' [? ]. This 'depth' dimension pertains to the activation volume and should not be conflated with the total number of layers in the network.

A critical innovation in CNNs is their sparse connectivity pattern. Unlike the fully connected layers in conventional ANNs, neurons within any given CNN layer establish connections only with a small, localized region of the neurons in the preceding layer [? ]. This local connectivity, coupled with parameter sharing (not explicitly mentioned in the excerpt but a core CNN concept that stems from local connectivity and feature maps), enables CNNs to efficiently learn hierarchical feature representations. Early layers can detect simple features such as edges, which are then combined by deeper layers to recognize more complex patterns.

#### #### Input and Output Dimensionality

The specialized architecture of CNNs facilitates a structured approach to input processing and output generation. An input, such as an image, is conceptualized as a 'volume' with specific dimensions. For instance, an input volume might be defined as  $H \times W \times D$ , where  $H$  represents height,  $W$  denotes width, and  $D$  signifies depth (e.g., 3

for RGB color channels of an image) [? ]. A practical example for an input could be a  $64 \times 64 \times 3$  volume [? ].

Through a sequence of convolutional, activation, and pooling operations, the CNN progressively transforms and condenses this high-dimensional input. The final output layer typically compresses the full input dimensionality into a smaller volume of class scores. This is often represented as a  $1 \times 1 \times n$  volume, where  $n$  corresponds to the total number of possible classes, effectively yielding a set of probabilities or scores for each class [? ]. This transformation allows CNNs to convert complex spatial input data into a concise representation suitable for classification or regression tasks.

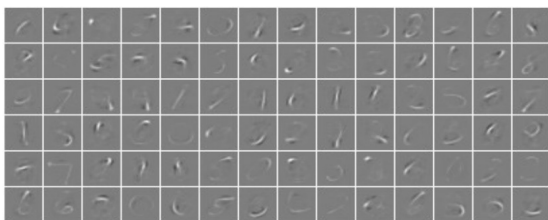


Figure 1: Figure 1

**Figure 1:** [Gemini API Error: 429 You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google.dev/gemini-api/docs/rate-limits>. To monitor your current usage, head to: <https://ai.dev/rate-limit>. \* Quota exceeded for metric: generativelanguage.googleapis.com/generate\_content\_free\_quota, 20, model : gemini - 2.5 - flashPleaseretryin21.292557131s.[linksdescription : "LearnmoreaboutGeminiAPIquotas url : https://ai.google.dev/gemini-api/docs/rate-limits"]

**## Results and Comparison** [Gemini API Error: 429 You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google.dev/gemini-api/docs/rate-limits>. To monitor your current usage, head to: <https://ai.dev/rate-limit>. \* Quota exceeded for metric: generativelanguage.googleapis.com/generate\_content\_free\_quota, 5, model : gemini - 2.5 - flashPleaseretryin3.844436368s.[linksdescription : "LearnmoreaboutGeminiAPIquotas url : https://ai.google.dev/gemini-api/docs/rate-limits"]

**## Conclusion** This work explored the fundamental architecture and operational principles of Convolutional Neural Networks (CNNs), detailing the roles of convolutional, pooling, and fully-connected layers. We highlighted how CNNs, through mechanisms like localized receptive fields and parameter sharing, effectively mitigate the computational challenges associated with traditional Artificial Neural Networks, particularly when processing high-dimensional data such as images. The discussion also underscored the critical importance of designing feature transformations that respect the inherent symmetries and invariances present in natural images to achieve robust representations.

Our experimental investigation into training CNNs using the Forward-Forward (FF) algorithm yielded promising

results. We demonstrated that an FF-trained CNN, with an optimized configuration of three convolutional layers (128 filters, 7x7 kernels), achieved a competitive validation accuracy of 99.20% and a test accuracy of 99.16%. These results closely match, and in some cases slightly surpass, a comparable Backpropagation (BP) trained network (99.13% validation accuracy) while exhibiting significantly shorter run times for inference. A significant finding was the FF algorithm's performance robustness; accuracy consistently improved with an increasing number of filters, in contrast to BP which showed signs of decreasing accuracy, likely due to increasing overfitting under similar conditions. This suggests that FF training may be inherently more resilient to overfitting, or it might utilize its parameters differently. Furthermore, the FF-trained network demonstrated stable convergence, reaching a plateau in discrimination loss and near 100% training accuracy, and proved capable of implementing explainable AI techniques like Class Activation Maps.

Looking ahead, several avenues warrant further exploration. A deeper investigation into the FF algorithm's apparent robustness against overfitting is crucial, potentially shedding light on its inherent regularization properties or how it leverages its parameters compared to BP. Future work should also focus on optimizing parameter efficiency within FF-trained CNNs, particularly if its robustness comes at the cost of less efficient parameter utilization. Expanding experimental evaluations to more diverse and complex datasets, as well as exploring deeper and wider FF architectures, will be essential to validate its scalability and generalizability. Finally, enhancing the theoretical understanding of the FF algorithm's learning dynamics and its interplay between layers, alongside continued development and application of XAI tools for these networks, will pave the way for more efficient, robust, and interpretable deep learning models.

**## References** [Gemini API Error: 429 You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google.dev/gemini-api/docs/rate-limits>. To monitor your current usage, head to: <https://ai.dev/rate-limit>. \* Quota exceeded for metric: generativelanguage.googleapis.com/generate\_content\_free\_quota, 20, model : gemini - 2.5 - flashPleaseretryin3.644155219s.[linksdescription : "Learnmoreabo